```python
def trap(height):
    left, right = 0, len(height) - 1
    left_max = right_max = 0
    water = 0

    while left < right:
        if height[left] < height[right]:
            if height[left] >= left_max:
                left_max = height[left]
            else:
                water += left_max - height[left]
            left += 1
        else:
            if height[right] >= right_max:
                right_max = height[right]
            else:
                water += right_max - height[right]
            right -= 1

    return water


# Example
print(trap([0,1,0,2,1,0,1,3,2,1,2,1]))
```

```
6
```

```python
def longest_palindrome(s):
    res = ""

    for i in range(len(s)):
        for l, r in [(i, i), (i, i+1)]:
            while l >= 0 and r < len(s) and s[l] == s[r]:
                if (r - l + 1) > len(res):
                    res = s[l:r+1]
                l -= 1
                r += 1

    return res


# Example
print(longest_palindrome("babad"))
```

```
bab
```

```python
def word_break(s, wordDict):
    dp = [False] * (len(s) + 1)
    dp[0] = True

    for i in range(1, len(s) + 1):
        for word in wordDict:
            if i >= len(word) and dp[i - len(word)]:
                if s[i-len(word):i] == word:
                    dp[i] = True
                    break

    return dp[-1]


# Example
print(word_break("applepenapple", ["apple", "pen"]))
```

```
True
```

```python
from collections import Counter
import heapq

def top_k_frequent(nums, k):
    freq = Counter(nums)  # count frequency
    return heapq.nlargest(k, freq.keys(), key=freq.get)


# Example
print(top_k_frequent([1,1,1,2,2,3], 2))
```

```
[1, 2]
```

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


def detect_cycle(head):
    slow = head
    fast = head

    while fast and fast.next:
        slow = slow.next          # move 1 step
        fast = fast.next.next     # move 2 steps

        if slow == fast:
            return True           # cycle detected

    return False                  # no cycle
```

```python
def min_platform(arr, dep):
    arr.sort()
    dep.sort()

    i = j = 0
    platforms = 0
    max_platforms = 0

    while i < len(arr) and j < len(dep):
        if arr[i] <= dep[j]:
            platforms += 1
            max_platforms = max(max_platforms, platforms)
            i += 1
        else:
            platforms -= 1
            j += 1

    return max_platforms


# Example
arr = [900, 940, 950, 1100, 1500, 1800]
dep = [910, 1200, 1120, 1130, 1900, 2000]

print(min_platform(arr, dep))
```

```
3
```

```python
from collections import deque

def max_sliding_window(nums, k):
    dq = deque()
    result = []

    for i in range(len(nums)):
        # Remove out-of-window elements
        while dq and dq[0] < i - k + 1:
            dq.popleft()

        # Remove smaller elements
        while dq and nums[dq[-1]] < nums[i]:
            dq.pop()

        dq.append(i)

        # Add result
        if i >= k - 1:
            result.append(nums[dq[0]])

    return result


# Example
print(max_sliding_window([1,3,-1,-3,5,3,6,7], 3))
```

```
[3, 3, 5, 5, 6, 7]
```

```python
from collections import defaultdict

def group_anagrams(strs):
    d = defaultdict(list)

    for word in strs:
        key = ''.join(sorted(word))   # sort letters
        d[key].append(word)

    return list(d.values())


# Example
print(group_anagrams(["eat","tea","tan","ate","nat","bat"]))
```

```
[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

```python
def search_rotated(nums, target):
    left = 0
    right = len(nums) - 1

    while left <= right:
        mid = (left + right) // 2

        if nums[mid] == target:
            return mid


        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1


        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1

    return -1
```

```python
def product_except_self(nums):
    n = len(nums)
    result = [1] * n

    # Prefix product
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]

    # Suffix product
    suffix = 1
    for i in range(n-1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]

    return result


# Example
nums = [1, 2, 3, 4]
print(product_except_self(nums))
```

```
[24, 12, 8, 6]
```

```python
def merge_intervals(intervals):
    if not intervals:
        return []


    intervals.sort(key=lambda x: x[0])


    merged = [intervals[0]]


    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:
            last[1] = max(last[1], current[1])
        else:
            merged.append(current)

    return merged

# Test with example
intervals = [[1,3], [2,6], [8,10]]
print(merge_intervals(intervals))
```

```
[[1, 6], [8, 10]]
```

```python
def merge_intervals(intervals):
    if not intervals:
        return []


    intervals.sort(key=lambda x: x[0])


    merged = [intervals[0]]


    for current in intervals[1:]:
        last = merged[-1]
        if current[0] <= last[1]:
            last[1] = max(last[1], current[1])
        else:
            merged.append(current)

    return merged

# Test with example
intervals = [[1,3], [2,6], [8,10]]
print(merge_intervals(intervals))
```

```
[[1, 6], [8, 10]]
```

```python
def length_of_longest_substring(s: str) -> int:
    last_seen = {}
    left = 0
    max_len = 0

    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        max_len = max(max_len, right - left + 1)

    return max_len

# Example
print(length_of_longest_substring("abcabcbb"))
print(length_of_longest_substring(""))
print(length_of_longest_substring("bbbbb"))
print(length_of_longest_substring("pwwkew"))
```

```
3
0
1
3
```

```python
def twoSum(nums, target):
    hash_map = {}

    for i, num in enumerate(nums):
        complement = target - num
        if complement in hash_map:
            return [hash_map[complement], i]
        hash_map[num] = i

    return []


nums = [2, 7, 11, 15]
target = 9
print(twoSum(nums, target))  # Output:
```

```
[0, 1]
```